

# Classe de testes unitários para classe: CartaoCredito

**Testador:** Eric Leal de Araujo Martins

- **Objetivo do teste: validar regras de negócio relacionadas à fatura, limite e cobrança de juros**

## Primeiro teste:

Testa se o método `creditar()` adiciona corretamente valores à fatura.

- Cenário 1: Crédito válido dentro do limite
  - É realizado um crédito de 300.0 em um cartão com fatura inicial 0.0.
  - O teste verifica se a fatura final é 300.0.

```
@Test
void deveAdicionarValorNaFatura() {
    cartao.creditar(300);
    assertEquals(expected: 300, cartao.getFatura());
}
```

## Segundo teste:

Testa se o sistema impede que a fatura ultrapasse o limite definido do cartão

- Cenário 1: Crédito até o limite máximo
  - É creditado o valor de 1000.0 (limite do cartão)
  - O teste verifica se a fatura atinge corretamente o limite.
- Cenário 2: Tentativa de ultrapassar o limite
  - Após atingir o limite, é feita uma nova tentativa de crédito de 100.0.
  - O teste valida que a fatura permanece em 1000.0, impedindo excesso.

```
@Test
void naoDeveUltrapassarOLimite() {
    cartao.creditar(1000);
    cartao.creditar(100); // tentativa de ultrapassar limite
    assertEquals(expected: 1000, cartao.getFatura());
}
```

### Terceiro teste:

Testa se o sistema reconhece corretamente o pagamento total da fatura.

- **Cenário 1: Quitação completa da fatura**

- É creditado o valor de 200.0 e, em seguida, aplicado um valor de -200.0 simulando pagamento total.
- O teste verifica se a fatura é reduzida corretamente.

```
@Test
void deveMarcarFaturaComoPagaQuandoZeradaOuNegativa() {
    cartao.creditar(200);
    cartao.setFatura(-200); // quitando a fatura
    assertEquals(expected: 0, cartao.getFatura());
}
```

### Quarto teste:

Testa se os juros são aplicados quando a fatura não foi paga e o mês atual é posterior ao mês da fatura

- **Cenário 1: Fatura não paga de mês anterior**

- É criado um cartão com fatura de 200.0, marcada como não paga e referente ao mês anterior.
- O método `cobrarJuros()` é executado.

- **Validação:**

O teste verifica se o valor da fatura após a operação é maior que o valor inicial, confirmando a aplicação de juros.

```

@Test
void deveAplicarJurosQuandoMesAvancaEFaturaNaoPaga() {
    int mesAtual = Ano.getInstancia().getIndexMesAtual();
    cartao = new CartaoCredito(
        200,
        mesAtual - 1, // mês anterior
        false,        // fatura não paga
        1000,
        1111,
        123
    );
    double antes = cartao.getFatura(); // guarda o valor da fatura antes de aplicar o juros
    cartao.cobrarJurus();
    assertTrue(cartao.getFatura() > antes); // verifica se ta maior o valor após a aplicação
}

```

### Quinto teste:

Testa se o sistema não aplica juros quando a fatura já foi quitada.

- **Cenário 1: Fatura paga em mês anterior**
  - É criado um cartão com fatura marcada como paga.
  - Mesmo com a passagem de mês, o método de juros é chamado.
- **Validação:**
  - O teste verifica se o valor da fatura permanece inalterado.

```

@Test
void naoDeveAplicarJurosSeFaturaJaFoiPaga() {
    int mesAtual = Ano.getInstancia().getIndexMesAtual();
    cartao = new CartaoCredito(
        200,
        mesAtual - 1,
        true, // fatura já paga
        1000,
        1111,
        123
    );

    double antes = cartao.getFatura(); // guarda o valor da fatura antes de aplicar
    cartao.cobrarJurus();
    assertEquals(antes, cartao.getFatura()); //verifica se foi cobrado juros da fatura já paga
}

```

### Sexto teste:

Testa se o sistema não aplica juros quando a fatura ainda está dentro do mesmo período.

- **Cenário 1: Fatura no mês atual**

- É criado um cartão com fatura não paga, porém no mês atual.
- O método de cobrança de juros é executado.

- **Validação:**

O teste garante que a fatura não sofre alteração, pois ainda não está em atraso.

```
@Test
void naoDeveAplicarJurosSeAindaForMesmoMes() {
    int mesAtual = Ano.getInstancia().getIndexMesAtual();
    cartao = new CartaoCredito(
        200,
        mesAtual, // mesmo mês
        false,
        1000,
        1111,
        123
    );

    double antes = cartao.getFatura(); // guarda o valor da fatura antes de aplicar
    cartao.cobrarJuros();
    assertEquals(antes, cartao.getFatura()); //verifica se foi cobrado juros da fatura atual
}
```

# Classe de testes unitários para classe: Conta

**Testador:** Italo Coelho Ide Chiaradia

- **Objetivo do teste: validar regras de negócio relacionadas a depósitos, saques, transferências, empréstimos e extrato, transação via pix.**

## Primeiro teste: Depósitos

Testa se o método depositar() adiciona corretamente um valor ao saldo e impede valores negativos.

- Cenário 1: Depositado 50.0 em uma conta com saldo 100.0. O teste verifica se o saldo final é 150.0.
- Cenário 2: Tentativa de depósito de -50.0. O teste garante que o saldo permaneça em 100.0.

```
// Testa se o método depositar adiciona corretamente um valor ao saldo.
@Test  Italo Chiaradia
void deveDepositarValorCorretamente() {
    conta.depositar( valor: 50.0);
    assertEquals( expected: 150.0, conta.getSaldo());
}

// Testa se o sistema impede depósitos com valores negativos.
@Test  Italo Chiaradia
void depositoComValorNegativoNaoEhValidado() {
    conta.depositar( valor: -50.0);

    // O saldo inicial é 100.0. Um depósito negativo não deveria ser processado.
    // O teste falhará aqui pois o comportamento atual aplica o valor e deixa o saldo em 50.0.
    assertEquals( expected: 100.0, conta.getSaldo(), (message: "BUG: O sistema não validou o depósito negativo e o saldo foi subtraído.");
}
}
```

## Segundo teste: Saques

Testa se o método sacar() subtrai valores corretamente e valida limites de saldo.

- Cenário 1: Sacado 30.0 de um saldo de 100.0. O teste verifica se restam 70.0.
- Cenário 2: Tentativa de saque de valor negativo (-30.0). O saldo deve permanecer inalterado.
- Cenário 3: Tentativa de saque de 200.0 (maior que o saldo). O sistema deve impedir que o saldo fique negativo.

```

// Testa se o método sacar subtrai corretamente um valor do saldo.
@Test  ⚡ Italo Chiaradia
void deveSacarValorCorretamente() {
    conta.sacar( valor: 30.0);
    assertEquals( expected: 70.0, conta.getSaldo());
}

// Testa se o método sacar retorna erro quando tenta sacar valor negativo.
@Test  ⚡ Italo Chiaradia
void deveRetornarErroAoSacarValorNegativo() {
    conta.sacar( valor: -30.0);
    assertEquals( expected: 100.0, conta.getSaldo());
}

// Testa se saque de valor maior que saldo é negado
@Test  ⚡ Italo Chiaradia
void saqueComValorMaiorQueSaldoNaoDeveSerPermitido() {
    // Saldo inicial é 100.0. Tentamos sacar 200.0.
    conta.sacar( valor: 200.0);

    // O saldo não pode ficar negativo se a conta não tiver limite de cheque especial.
    assertTrue( condition: conta.getSaldo() >= 0.0, message: "BUG: O sistema permitiu sacar mais do que o saldo disponível.");
}

```

### Terceiro teste: Transações via Pix

Este módulo valida a integridade das transferências instantâneas, garantindo que o dinheiro saia da origem, chegue ao destino e que as travas de segurança funcionem corretamente.

- Cenário 1: Crédito na conta destino  
Verifica se, ao realizar um Pix de 50.0, o saldo da conta que recebe o dinheiro é atualizado corretamente (de 0.0 para 50.0).
- Cenário 2: Débito na conta origem  
Garante que o valor enviado seja subtraído do saldo do remetente (de 100.0 para 50.0).
- Cenário 3: Pix para a mesma conta  
Testa a regra de negócio que impede transferências para si mesmo. O saldo deve permanecer inalterado (100.0).
- Cenário 4: Valores negativos  
Valida se o sistema bloqueia transferências com valores abaixo de zero. O teste monitora tanto a conta de origem quanto a de destino para garantir que nenhum saldo seja corrompido.

- Cenário 5: Conta destino nula

Verifica se o sistema trata corretamente a ausência de um destinatário válido, lançando uma `IllegalArgumentException` em vez de um erro genérico do sistema.

- Cenário 6: Saldo insuficiente

Garante que o cliente não consiga transferir um valor (ex: 150.0) maior do que o saldo disponível em conta (100.0), impedindo que o saldo fique negativo indevidamente.

```
// Testa se a transferência via Pix credita corretamente o valor na conta destino.
@Test  ⚡ Italo Chiaradia
void deveTransferirPixParaOutraConta() {
    Conta contaDestino = new ContaFake( numeroConta: 2, saldo: 0.0, cartao: null, divida: 0.0);
    conta.fazerPix(contaDestino, valor: 50.0);
    assertEquals( expected: 50.0, contaDestino.getSaldo());
}

// Testa se a transferência via Pix debita corretamente o valor na conta origem.
@Test  ⚡ Italo Chiaradia
void deveDebitarDinheiroEnviadoNaContaOrigem() {
    Conta contaDestino = new ContaFake( numeroConta: 2, saldo: 0.0, cartao: null, divida: 0.0);
    conta.fazerPix(contaDestino, valor: 50.0);
    assertEquals( expected: 50.0, conta.getSaldo());
}

// Testa se a transferência via Pix para mesma conta esta sendo permitido
@Test  ⚡ Italo Chiaradia
void pixParaContaPessoalDeveSerProibido() {
    conta.fazerPix(conta, valor: 50.0);
    assertEquals( expected: 100.0, conta.getSaldo());
}
```

```
// Testa se o sistema impede que um Pix seja feito com valor negativo.
@Test  @ Italo Chiaradia
void pixComValorNegativoNaoDeveSerPermitido() {
    Conta contaDestino = new ContaFake( numeroConta: 2, saldo: 0.0, cartao: null, divida: 0.0);
    conta.fazerPix(contaDestino, valor: -50.0);

    // O saldo da conta origem (100.0) não pode ser alterado
    assertEquals( expected: 100.0, conta.getSaldo(), message: "BUG: A conta origem teve o saldo alterado por um Pix negativo.");

    // O saldo da conta destino (0.0) também não pode receber o valor negativo
    assertEquals( expected: 0.0, contaDestino.getSaldo(), message: "BUG: A conta destino recebeu um Pix com valor negativo.");
}

// Teste para quando é passado um valor nulo no método fazerPix
@Test  @ Italo Chiaradia
void pixParaContaNulaDeveLancarExcecao() {
    // Se passarmos null, o sistema atual lança NullPointerException quando tenta acessar conta.saldo.
    // O correto seria validar antes e lançar IllegalArgumentException.
    assertThrows(IllegalArgumentException.class, () -> {
        conta.fazerPix( conta: null, valor: 50.0);
    }, message: "BUG: O sistema não validou a conta destino nula (vai estourar NullPointerException no código real).");
}
```

```
// Testa se o sistema impede que um Pix seja feito caso o valor exceda o saldo da conta origem.
@Test  @ Italo Chiaradia
void pixComValorMaiorQueSaldoNaoDeveSerPermitido() {
    Conta contaDestino = new ContaFake( numeroConta: 2, saldo: 0.0, cartao: null, divida: 0.0);
    // Saldo inicial é 100.0. Tentamos enviar 150.0.
    conta.fazerPix(contaDestino, valor: 150.0);

    // Assumindo que a regra de negócio não permite Pix usando limite especial para ficar negativo.
    assertTrue( condition: conta.getSaldo() >= 0.0, message: "BUG: O sistema permitiu transferir um valor de Pix maior que o saldo disponível.");
}
```

## Quarto teste: Gestão de Empréstimo

Valida a criação de dívida ao solicitar crédito ao banco.

- Cenário 1: Testa se a dívida aumenta corretamente ao solicitar 100.0 (somado aos 50.0 iniciais, totalizando 150.0).
- Cenário 2: Verifica se o sistema ignora solicitações de empréstimo com valores negativos, mantendo a dívida original.

```
// Testa se o método de requisitar empréstimo aumenta a dívida da conta.
@Test  @ Italo Chiaradia
void deveAumentarDividuaAoRequisitarEmprestimo() {
    conta.requisitarEmprestimo( valor: 100.0);
    assertEquals( expected: 150.0, conta.getDividaDeEmprestimo());
}

// Testa se o método de requisitar empréstimo diminui ao passar valor negativo
@Test  @ Italo Chiaradia
void requisitarEmprestimoDeValorNegativoEhInvalido() {
    conta.requisitarEmprestimo( valor: -100.0);
    assertEquals( expected: 50.0, conta.getDividaDeEmprestimo());
}
```

## Quinto teste: Pagar parcela do empréstimo

Validar se o pagamento de parcelas reduz corretamente o saldo devedor e se o sistema possui travas para valores inválidos ou excedentes.

- Cenário 1: Redução correta da dívida
  - Testa se o método `pagarParcelaDeEmprestimo()` subtrai o valor pago do montante total da dívida.
  - Execução: É realizado um pagamento de 20.0 para uma dívida inicial de 50.0.
  - Validação: O teste verifica se o saldo devedor restante é exatamente 30.0.
- Cenário 2: Bloqueio de valores negativos
  - Testa se o sistema impede que pagamentos negativos sejam processados, o que poderia aumentar indevidamente a dívida do cliente.
  - Execução: Tentativa de pagar o valor de -20.0.
  - Validação: O teste garante que a dívida permaneça em 50.0, confirmando que o valor negativo não foi aplicado.
- Cenário 3: Impedimento de dívida negativa (Pagamento Excedente)
  - Testa se o sistema impede que o saldo devedor fique abaixo de zero caso o pagamento seja maior que a dívida atual.
  - Execução: Tentativa de pagar 100.0 para uma dívida que é de apenas 50.0.
  - Validação: O teste verifica se a dívida final é maior ou igual a 0.0, garantindo que o sistema não gere um saldo credor indevido na rubrica de empréstimos.

```
// Testa se o pagamento de uma parcela reduz corretamente o valor da dívida de empréstimo.
@Test  * Italo Chiaradia
void deveDiminuirDívidaAoPagarParcela() {
    conta.pagarParcelaDeEmprestimo( valor: 20.0);
    assertEquals( expected: 30.0, conta.getDividaDeEmprestimo());
}

// Testa se o sistema barra pagamentos de parcela com valores negativos.
@Test  * Italo Chiaradia *
void pagarParcelaComValorNegativoNaoDeveSerValidado() {
    conta.pagarParcelaDeEmprestimo( valor: -20.0);
    assertEquals( expected: 50.0, conta.getDividaDeEmprestimo(), message: "BUG: O pagamento negativo foi processado e aumentou a dívida.");
}

// Testa se o sistema permite pagar um valor maior que o da parcela
@Test  * Italo Chiaradia *
void naoDevePermitirPagamentoMaiorQueADividida() {
    conta.pagarParcelaDeEmprestimo( valor: 100.0);
    assertTrue( condition: conta.getDividaDeEmprestimo() >= 0.0, message: "BUG: A dívida ficou negativa após um pagamento excedente.");
}
```

### **Sexto teste: Cobrança de Juros**

Validar a aplicação de taxas sobre o saldo devedor e garantir que não ocorram cobranças indevidas em contas sem dívidas ou com saldo negativo.

- Cenário 1: Aplicação de juros em dívida ativa
  - Testa se o método `cobrarJurusEmprestimo()` é executado corretamente quando o cliente possui uma dívida pendente.
  - Execução: O sistema identifica a dívida atual e aplica a taxa correspondente.
  - Validação: O teste verifica se o valor da dívida após a cobrança é diferente do valor inicial (validando o processamento do encargo).
- Cenário 2: Isenção para dívida zerada
  - Verifica se o sistema ignora a cobrança de juros quando o saldo devedor é inexistente.
  - Execução: O método de cobrança é chamado em uma conta com dívida igual a 0.0.
  - Validação: O teste garante que o valor da dívida permaneça em 0.0, assegurando que não houve cobrança indevida.
- Cenário 3: Proteção contra dívida negativa
  - Testa a robustez do sistema caso a dívida esteja negativa (ex: por um pagamento excedente), garantindo que juros não sejam aplicados sobre créditos.
  - Execução: O método é acionado em uma conta que possui uma "dívida" de -10.0.
  - Validação: O teste confirma que o valor permanece exatamente em -10.0, sem alterações por algoritmos de juros.

```

// Testa se os juros são aplicados corretamente quando a conta possui uma dívida de empréstimo ativa.
@Test  * Italo Chiaradia
void deveCobrarJurosQuandoDívidaMaiorQueZero() {
    double dividaInicial = conta.getDividaDeEmprestimo();
    conta.cobrarJurosEmprestimo();
    assertTrue( condition: conta.getDividaDeEmprestimo() < dividaInicial);
}

// Testa se NÃO são cobrados juros quando a conta não possui dívida de empréstimo.
@Test  * Italo Chiaradia
void naoDeveCobrarJurosQuandoDívidaZero() {
    Conta contaSemDivida = new ContaFake( numeroConta: 3, saldo: 100.0, cartao: null, divida: 0.0);
    contaSemDivida.cobrarJurosEmprestimo();
    assertEquals( expected: 0.0, contaSemDivida.getDividaDeEmprestimo());
}

// Testa se o sistema ignora a cobrança de juros caso a dívida, por algum motivo de falha, esteja negativa.
@Test  * Italo Chiaradia *
void naoDeveCobrarJurosQuandoDívidaForNegativa() {
    Conta contaDividaNegativa = new ContaFake( numeroConta: 4, saldo: 100.0, cartao: null, divida: -10.0);
    contaDividaNegativa.cobrarJurosEmprestimo();
    assertEquals( expected: -10.0, contaDividaNegativa.getDividaDeEmprestimo());
}

```

### Sétimo teste: setExtrato

Validar a integridade do histórico de transações da conta, garantindo que novas movimentações sejam registradas corretamente e que objetos inválidos sejam rejeitados.

- Cenário 1: Registro de nova movimentação
  - Testa se o método setExtrato() adiciona um objeto de movimentação à lista de histórico da conta.
  - Execução: É criada uma nova Movimentacao de 100.0 (Tipo: ENTRADA) e enviada para o método.
  - Validação: O teste verifica se o tamanho da lista de extrato da conta passou a ser 1, confirmando o armazenamento.
- Cenário 2: Proteção contra objetos nulos
  - Verifica se o sistema possui uma trava de segurança para impedir a corrupção do histórico com dados nulos.
  - Execução: O método setExtrato() é chamado passando um valor null.
  - Validação: O teste espera que o sistema lance uma IllegalArgumentException. Caso o sistema aceite o valor nulo ou lance um erro genérico (como NullPointerException), o teste sinaliza um erro de validação (BUG).

```
// Testa se uma nova movimentação é adicionada corretamente ao extrato da conta.
@Test  ⚡ Italo Chiaradia
void deveAdicionarMovimentacaoAoExtrato() {
    Movimentacao mov = new Movimentacao(
        dinheiroMovimentado: 100.0,
        Movimentacao.TipoDaMovimentacao.ENTRADA
    );
    conta.setExtrato(mov);
    assertEquals( expected: 1, conta.getExtrato().size());
}

// Testa se o método de registrar extrato não quebra recebendo um objeto nulo.
@Test  ⚡ Italo Chiaradia
void extratoComMovimentacaoNulaDeveLancarExcecao() {
    assertThrows(IllegalArgumentException.class, () -> {
        conta.setExtrato(null);
    }, message: "BUG: O sistema aceitou adicionar uma movimentação nula ao extrato.");
}
```

# **Classe de testes unitários para classe:**

## **ContaPoupanca**

**Testador:** Gabriel Abreu Alves da Silva e Silva

- **Objetivo do teste: validar regras de negócio relacionadas a rendimentos, informe de rendimentos, tipo de conta e movimentação bancária.**

### **Primeiro teste: acrescentarRendimento**

Testa se o método `acrescentarRendimento()` aumenta corretamente o saldo da conta aplicando a taxa de rendimento mensal da poupança e registra a movimentação.

- Cenário 1: Rendimento com saldo positivo padrão:
  - Uma conta com saldo de 1000.0 recebe rendimento aplicando a fórmula:  $\text{saldo} + (\text{saldo} / 0.8)$
  - O teste verifica se o saldo final é  $1000.0 + (1000.0 / 0.8) = 2250.0$
  - Também valida se uma movimentação de tipo ENTRADA é criada no informe de rendimento
- Cenário 2: Rendimento com saldo ZERO:
  - Uma conta com saldo 0.0 recebe rendimento aplicando a fórmula:  $0.0 + (0.0 / 0.8)$
  - O teste verifica se o saldo final permanece 0.0
  - Valida comportamento de borda ao aplicar rendimento com saldo nulo
- Cenário 3: Rendimento com saldo MUITO GRANDE:
  - Uma conta com saldo de 1.000.000,0 recebe rendimento
  - O teste verifica se o cálculo é aplicado corretamente para valores muito grandes:  $1000000.0 + (1000000.0 / 0.8) = 2.250.000,0$
  - Valida que o sistema não possui limitações ou erros de overflow para saldos elevados

- Cenário 4: Rendimento com saldo NEGATIVO:
  - Uma conta com saldo de -500.0 recebe rendimento aplicando a fórmula:  $-500.0 + (-500.0 / 0.8)$
  - O teste verifica se o saldo final é  $-500.0 + (-625.0) = -1125.0$
  - Valida comportamento de borda ao aplicar rendimento em conta com saldo negativo
  - Não deveria ser possível aplicar rendimento em conta poupança com saldo negativo
  
- Cenário 5: Múltiplos rendimentos consecutivos:
  - Uma conta recebe três chamadas consecutivas de `acrescentarRendimento()`
  - O teste verifica se o informe de rendimento acumula 3 movimentações
  - Valida comportamento de múltiplas operações sem perda de dados

### **Segundo teste: setInformeRendimento**

Testa se o método `setInformeRendimento()` adiciona corretamente uma movimentação à lista de informe de rendimento.

- Cenário 1: Adicionar movimentação ao informe vazio:
  - Uma conta possui lista de informe de rendimento vazia
  - Uma movimentação de 100.0 com tipo ENTRADA é adicionada
  - O teste verifica se o tamanho da lista é 1
  - Valida que a movimentação foi armazenada corretamente

### **Terceiro teste: getInformeRendimento**

Testa se o método `getInformeRendimento()` retorna a lista de movimentações de rendimento corretamente.

- Cenário 1: Lista vazia no início:
  - Uma nova conta poupança é criada
  - O teste verifica se `getInformeRendimento()` retorna uma lista vazia
  - Valida inicialização correta da lista no construtor

#### **Quarto teste: getTipoConta**

Testa se o método `getTipoDaConta()` retorna corretamente o tipo de conta.

- Cenário 1: Identificação de conta poupança:
  - Uma instância de `ContaPoupanca` chama o método `getTipoDaConta()`
  - O teste verifica se o retorno é exatamente a string "Poupanca"
  - Valida a identificação correta do tipo de conta

#### **Quinto teste: movimentacaoBancaria**

Testa se o método `movimentacaoBancaria()` registra corretamente as despesas no banco.

- Cenário 1: Registro de despesa simples:
  - Uma conta chama `movimentacaoBancaria(100.0)`
  - O teste verifica se as despesas do Banco aumentam de forma correspondente
  - Valida integração com a classe Banco (Singleton)
- Cenário 2: Movimentação com valor ZERO:
  - Uma conta chama `movimentacaoBancaria(0.0)`
  - O teste verifica se as despesas não são alteradas
  - Valida comportamento de borda com valor nulo
- Cenário 3: Múltiplas movimentações acumuladas:
  - Uma conta realiza três chamadas sucessivas:  
`movimentacaoBancaria(50.0)`, `movimentacaoBancaria(75.0)` e `movimentacaoBancaria(100.0)`
  - O teste verifica se as despesas totais do Banco somam 225.0 (50 + 75 + 100)
  - Valida acúmulo correto de múltiplas operações

# Classe de testes unitários para classe: ClienteWinx

**Testador:** João Victor de Oliveira Costa

- **Objetivo do teste: validar regras de negócio relacionadas a acúmulo de pontos de compra e conversão desses pontos em saldo na conta, garantindo também o registro correto no extrato.**

## Primeiro teste: Pontos de compra (obterPontosDeCompra)

Testa se o método obterPontosDeCompra() incrementa corretamente os pontos do cliente.

Cenário 1: Cliente inicia com 0 pontos. Ao chamar obterPontosDeCompra() uma vez, o teste verifica se os pontos passam a ser 1.

```
@Test
void obterPontosDeCompra_incrementaEm1() {
    cliente.obterPontosDeCompra();
    assertEquals(expected: 1, actual: cliente.getPontosDeCompra());
}
```

Cenário 2: Cliente inicia com 0 pontos. Ao chamar obterPontosDeCompra() 5 vezes, o teste verifica se os pontos passam a ser 5 (acúmulo correto).

```
@Test
void converterPontosEmSaldo_convertZeroPontosERregistraExtrato() {
    for (int i = 0; i < 5; i++) {
        cliente.obterPontosDeCompra();
    }
    assertEquals(expected: 5, actual: cliente.getPontosDeCompra());
}
```

## Segundo teste: Converter pontos em saldo (converterPontosEmSaldo)

Testa se o método converterPontosEmSaldo(Conta conta) converte pontos em saldo corretamente, zera os pontos e registra a movimentação no extrato.

```

@Test
void converterPontosEmSaldo_converteZeraPontosERegistraExtrato() {
    for (int i = 0; i < 5; i++) {
        cliente.obterPontosDeCompra();
    }
    assertEquals(expected: 5, actual: cliente.getPontosDeCompra());
    cliente.converterPontosEmSaldo(conta);

    assertEquals(expected: 0, actual: cliente.getPontosDeCompra());
    assertEquals(expected: 115.0, actual: conta.getSaldo(), delta: 0.0001);

    assertFalse(condition: conta.getExtrato().isEmpty());
    Movimentacao ultima = conta.getExtrato().get(index: conta.getExtrato().size() - 1);
    assertEquals(expected: Movimentacao.TipoDaMovimentacao.ENTRADA, actual: ultima.getTipoDaMovimentacao());
    assertEquals(expected: 15.0, actual: ultima.getDinheiroMovimentado(), delta: 0.0001);
}

```

Cenário 1: Conversão correta + zero pontos

Execução: Cliente acumula 5 pontos e chama converterPontosEmSaldo() em uma conta com saldo inicial 100.0.

Validação: O teste verifica:

pontos = 0 após a conversão

saldo final = 115.0 (pois 5 pontos × bônus 3 = 15)

Cenário 2: Registro no extrato

Execução: Após a conversão, o teste verifica o extrato da conta.

Validação: O teste garante que foi adicionada uma Movimentacao no extrato com:

Tipo = ENTRADA

Valor movimentado = 15.0

### Terceiro teste: Converter pontos em saldo com pontos negativos

Testa o método converterPontosEmSaldo() quando o cliente possui uma quantidade inválida de pontos (negativa), verificando se ocorre conversão negativa e impacto no saldo.

```

@Test
void converterPontosEmSaldo_comPontosNegativos() {
    ClienteWinx clienteComPontosNegativos = new ClienteWinx(nome: "Ana", cpf: "123", pontosDeCompra: -10);

    double saldoAntes = conta.getSaldo();
    clienteComPontosNegativos.converterPontosEmSaldo(conta);

    assertTrue(condition: saldoAntes > conta.getSaldo(),
        message: "BUG: pontos negativos geraram conversão negativa e diminuíram o saldo.");
}

```

Cenário 1: Pontos negativos geram conversão negativa (BUG / fragilidade)

Execução: é criado um cliente com `pontosDeCompra = -10` e uma conta com saldo inicial 100.0. Em seguida, chama-se `converterPontosEmSaldo(conta)`.

Regra/impacto esperado pelo teste atual: o saldo final da conta deve ficar menor do que o saldo inicial (indicando que a conversão foi negativa).

Validação: o teste compara `saldoAntes` com o saldo após a conversão e garante que `saldoAntes > conta.getSaldo()`.

Observação (importante):

Esse teste não valida uma regra “correta” do negócio; ele documenta um comportamento perigoso do sistema (BUG).

Pela lógica atual, a conversão é calculada como:

$\text{saldoConvertido} = \text{pontosDeCompra} \times 3 = -10 \times 3 = -30$   
e isso reduz o saldo da conta (100.0 → 70.0).

Se a regra do sistema for “pontos nunca podem ser negativos”, então o correto seria o método bloquear essa situação (ex: lançar `IllegalArgumentException` ou tratar pontos negativos como 0). Nesse caso, o teste deveria ser reescrito para esperar a trava, e não para confirmar a redução do saldo.

# Classe de testes unitários para classe: ContaCorrente

**Testador:** Arthur Jardim Antunes Ferreira

- **Objetivo do teste:** Validar o comportamento da classe ContaCorrente, garantindo que operações como pagamento de fatura, taxas, movimentações bancárias e operações básicas (depositar, sacar e pix) funcionem corretamente de acordo com as regras de negócio.

## Testes do método pagarFatura

### Primeiro teste:

Testa se o método pagarFatura() reduz corretamente o saldo da conta  
Foi pago 200 da fatura e comparado o saldo, que passou de 1000 para 800

```
@Test
@DisplayName("pagarFatura: deve deduzir o valor do saldo")
void pagarFatura_deveReduzirSaldo() {
    contaCorrente.pagarFatura(valor: 200.0);

    assertEquals(800.0, contaCorrente.getSaldo(), 0.001);
}
```

### Segundo teste:

Testa se o pagamento da fatura reduz o valor da fatura do cartão de crédito  
Foi simulada uma fatura de 300 no cartão e depois pago o mesmo valor  
O teste verifica se a fatura foi zerada

```
@DisplayName("pagarFatura: deve reduzir a fatura do cartão de crédito")
void pagarFatura_deveReduzirFaturaDoCartaoCredito() {
    cartaoCredito.setFatura(valor: 300.0); // simula uma compra no crédito
    contaCorrente.pagarFatura(valor: 300.0);

    assertEquals(0.0, cartaoCredito.getFatura(), 0.001);
}
```

### Terceiro teste:

Testa se pagar valor zero não altera o saldo

Foi feito pagamento de 0 e verificado se o saldo continua 1000

```
@DisplayName("pagarFatura: valor zero não altera o saldo")
void pagarFatura_valorZeroNaoAlteraSaldo() {
    contaCorrente.pagarFatura(valor: 0.0);

    assertEquals(SALDO_INICIAL, contaCorrente.getSaldo(), 0.001);
}
```

### Quarto teste:

Testa o comportamento ao pagar um valor maior que o saldo

Foi pago 1500 com saldo inicial de 1000

O teste verifica se o saldo fica negativo (-500)

```
@DisplayName("pagarFatura: valor maior que o saldo deixa saldo negativo")
void pagarFatura_valorMaiorQueOSaldoResultaEmSaldoNegativo() {
    contaCorrente.pagarFatura(valor: 1500.0);

    assertEquals(-500.0, contaCorrente.getSaldo(), 0.001);
}
```

## Testes do método **descontarTaxa**

### Primeiro teste:

Testa se a taxa de manutenção é descontada do saldo

Foi chamado o método e verificado se o saldo diminuiu conforme a taxa definida

```
@DisplayName("descontarTaxa: deve deduzir a taxa de manutenção do saldo")
void descontarTaxa_deveReduzirSaldoPelaTaxa() {
    double taxaEsperada = Conta.taxaManutencaoConta;

    contaCorrente.descontarTaxa();

    assertEquals(SALDO_INICIAL - taxaEsperada, contaCorrente.getSaldo(), 0.001);
}
```

### Segundo teste:

Testa se a taxa gera registro no extrato

Após chamar o método, o teste verifica se o extrato não está vazio

```

@DisplayName("descontarTaxa: deve registrar movimentação no extrato")
void descontarTaxa_deveAdicionarMovimentacaoNoExtrato() {
    contaCorrente.descontarTaxa();

    assertFalse(contaCorrente.getExtrato().isEmpty());
}

```

### Terceiro teste:

Testa se o valor da taxa é adicionado às receitas do banco

Foi comparado o valor das receitas antes e depois da execução

```

@DisplayName("descontarTaxa: deve repassar a taxa às receitas do banco")
void descontarTaxa_deveAumentarReceitasDoBanco() {
    double receitasAntes = Banco.getInstancia().getReceitas();

    contaCorrente.descontarTaxa();

    assertTrue(Banco.getInstancia().getReceitas() > receitasAntes);
}

```

## Testes do método `movimentacaoBancaria`

### Primeiro teste:

Testa se valores positivos aumentam as receitas do banco

Foi feita uma movimentação de 500 e verificado se o banco recebeu esse valor

```

@DisplayName("movimentacaoBancaria: deve aumentar as receitas do banco")
void movimentacaoBancaria_deveAumentarReceitasDoBanco() {
    contaCorrente.movimentacaoBancaria(valor: 500.0);

    assertEquals(500.0, Banco.getInstancia().getReceitas(), 0.001);
}

```

### Segundo teste:

Testa se valores negativos não alteram as receitas

Foi feita uma movimentação de -100

O teste verifica se as receitas continuam 0

```
@DisplayName("movimentacaoBancaria: valor negativo não altera receitas")
void movimentacaoBancaria_valorNegativoNaoAlteraReceitas() {
    contaCorrente.movimentacaoBancaria(-100.0);

    assertEquals(0.0, Banco.getInstancia().getReceitas(), 0.001);
}
```

## Teste do método `getCartaoCredito`

### Teste único:

Testa se o cartão de crédito retornado é o mesmo associado à conta  
O teste verifica se a referência do objeto é a mesma criada no início

```
@DisplayName("getCartaoCredito: deve retornar o cartão de crédito associado")
void getCartaoCredito_deveRetornarOCartaoCorreto() {
    assertEquals(cartaoCredito, contaCorrente.getCartaoCredito());
}
```

## Teste do método `getTipoDaConta`

### Teste único:

Testa se o tipo da conta é retornado corretamente  
O método deve retornar a string "Corrente"

```
@DisplayName("getTipoDaConta: deve retornar 'Corrente'")
void getTipoDaConta_deveRetornarStringCorreta() {
    assertEquals("Corrente", contaCorrente.getTipoDaConta());
}
```

## Testes do construtor e getters

### Primeiro teste:

Testa se o número da conta foi inicializado corretamente  
Verifica se o valor retornado é 1001

```
@DisplayName("Construtor: deve inicializar o número da conta corretamente")
void construtor_deveInicializarNumeroDaConta() {
    assertEquals(NUMERO_CONTA, contaCorrente.getNumeroConta());
}
```

### Segundo teste:

Testa se o saldo inicial foi definido corretamente

Verifica se o saldo é 1000

```
@DisplayName("Construtor: deve inicializar o saldo corretamente")
void construtor_deveInicializarSaldo() {
    assertEquals(SALDO_INICIAL, contaCorrente.getSaldo(), 0.001);
}
```

### Terceiro teste:

Testa se a dívida inicial está correta

Verifica se o valor é 0

```
@DisplayName("Construtor: deve inicializar a dívida de empréstimo corretamente")
void construtor_deveInicializarDividaDeEmprestimo() {
    assertEquals(DIVIDA, contaCorrente.getDividaDeEmprestimo(), 0.001);
}
```

## Testes de operações básicas

### Primeiro teste:

Testa se o método depositar() aumenta o saldo

Foi depositado 500 e o saldo passou para 1500

```
@DisplayName("depositar: deve aumentar o saldo corretamente")
void depositar_deveAumentarOSaldo() {
    contaCorrente.depositar(valor: 500.0);

    assertEquals(1500.0, contaCorrente.getSaldo(), 0.001);
}
```

### Segundo teste:

Testa se o método sacar() reduz o saldo

Foi sacado 300 e o saldo passou para 700

```

@DisplayName("sacar: deve reduzir o saldo corretamente")
void sacar_deveReduzirOSaldo() {
    contaCorrente.sacar(valor: 300.0);

    assertEquals(700.0, contaCorrente.getSaldo(), 0.001);
}

```

### Terceiro teste:

Testa se o método fazerPix() transfere valor entre contas

Foi feito um pix de 400 para outra conta

O teste verifica se a conta destino recebeu o valor

```

@DisplayName("fazerPix: deve transferir o valor para outra conta")
void fazerPix_deveTransferirValorParaOutraConta() {
    ContaCorrente contaDestino = new ContaCorrente(
        numeroConta: 2002, saldo: 0.0, new Cartao(numero: 99999, csv: 999),
        dividaDeEmprestimo: 0.0, new CartaoCredito(numero: 99999, csv: 999)
    );

    contaCorrente.fazerPix(contaDestino, valor: 400.0);

    assertEquals(400.0, contaDestino.getSaldo(), 0.001);
}

```